ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Khai thác dữ liệu và ứng dụng

Report Lab 02: Khai thác tập phổ biến

Thuật toán: HMine

Giảng viên hướng dẫn: GS.TS Lê Hoài Bắc
ThS. Lê Nhựt Nam
ThS. Nguyễn Ngọc Đức

Sinh viên: Phạm Khánh Linh MSSV: 23127083
Võ Trung Hiếu MSSV: 23127190
Nguyễn Thành Lợi MSSV: 23127408
Lớp: 23KHDL2

THÀNH PHỐ HỒ CHÍ MINH, 14 THÁNG 04 2026

Mục lục

1 Nền tảng lý thuyết **1**
1.1 Bài toán frequent itemset mining . . . 1
1.1.1 Cơ sở dữ liệu giao dịch . . . 1
1.1.2 Độ hỗ trợ . . . 1
1.1.3 Tập phổ biến . . . 2
1.1.4 Tập đóng và tập tối đại . . . 2
1.1.5 Tính chất Apriori . . . 3
1.2 Phân tích thuật toán H-Mine . . . 4
1.2.1 Ý tưởng cốt lõi . . . 4
1.2.2 Cấu trúc dữ liệu chính . . . 4
1.2.3 Mã giả thuật toán . . . 7
1.2.3.1 Thuật toán chính H-Mine . . . 7
1.2.3.2 Thủ tục đệ quy . . . 7
1.2.3.3 Giải thích ý nghĩa các bước. . . . 8
1.2.4 Phân tích độ phức tạp . . . 8
1.2.4.1 Độ phức tạp thời gian . . . 9
1.2.4.2 Độ phức tạp không gian . . . 9
1.2.4.3 Các yếu tố ảnh hưởng chính đến hiệu năng . . . 9
1.2.5 So sánh lịch sử . . . 10
1.2.5.1 Hạn chế của các thuật toán trước: . . . 10
1.2.5.2 Hạn chế còn lại . . . 11

2 Ví dụ minh họa tay **11**
2.1 Ví dụ 1: Cơ sở . . . 11
2.1.1 Bước 1: đếm support 1-itemset và xây dựng F-list . . . 12
2.1.2 Bước 2: tạo frequent-item projection . . . 12
2.1.3 Bước 3: khai phá nhánh chứa item a . . . 13
2.1.3.1 Nhánh ac . . . 14
2.1.3.2 Nhánh ad . . . 14
2.1.3.3 Nhánh ae . . . 14
2.1.4 Bước 4: khai phá các nhánh còn lại . . . 15
2.1.4.1 Nhánh c nhưng không chứa a . . . 15
2.1.4.2 Nhánh d nhưng không chứa a, c . . . 15
2.1.4.3 Nhánh e nhưng không chứa a, c, d . . . 16

2.1.4.4 Nhánh g 16
2.1.5 Kết quả cuối cùng 16
2.1.6 Kiểm tra chéo bằng liệt kê thủ công 16
2.2 Ví dụ 2: Tình huống đặc biệt 17
2.2.1 Bước 1: đếm support 1-itemset và xây dựng F-list 17
2.2.2 Bước 2: tạo frequent-item projection 17
2.2.3 Bước 3: khai phá nhánh chứa item a 18
2.2.3.1 Nhánh ab 19
2.2.3.2 Nhánh abc 19
2.2.3.3 Nhánh ac 19
2.2.3.4 Nhánh ad 20
2.2.4 Bước 4: khai phá các nhánh còn lại 20
2.2.4.1 Nhánh b nhưng không chứa a 20
2.2.4.2 Nhánh c nhưng không chứa a, b 20
2.2.4.3 Nhánh d 20
2.2.5 Kết quả cuối cùng 21
2.2.6 Kiểm tra chéo bằng liệt kê thủ công 21
2.2.7 Vì sao đây là tình huống đặc biệt quan trọng? 21

3 Cài đặt **22**
3.1 Môi trường và công cụ 22
3.2 Cấu trúc dữ liệu chính 22
3.2.1 HEntry 22
3.2.2 HeaderTable 23
3.3 Quy trình cài đặt bản cơ bản 23
3.4 Quy trình cài đặt bản tối ưu 24
3.4.1 Tối ưu truy cập và cấp phát bộ nhớ 24
3.4.2 Tích hợp cơ chế chuyển sang FP-tree trên dữ liệu dày 24
3.5 Đầu vào và đầu ra 24
3.5.1 Đọc dữ liệu 24
3.5.2 Ghi kết quả 24
3.6 Kiểm thử tính đúng đắn 25

4 Thực nghiệm và đánh giá **25**
4.1 Tập dữ liệu benchmark 25
4.2 Thiết lập thực nghiệm 26

4.3 Kiểm tra tính đúng đắn 27
4.4 Thời gian chạy theo minsup 28
4.5 Số lượng frequent itemset theo minsup 29
4.6 Sử dụng bộ nhớ 31
4.7 Khả năng mở rộng 32
4.8 Ảnh hưởng của độ dài giao dịch trung bình 33
4.9 Phân tích kết quả tổng hợp 34
4.10 Điểm mạnh, điểm yếu của thuật toán và hướng tối ưu tiếp theo 35
4.10.1 Điểm mạnh của thuật toán H-Mine 35
4.10.2 Điểm yếu của thuật toán H-Mine 35
4.10.3 Hai hướng tối ưu cụ thể có thể áp dụng tiếp theo 36

5 Ứng dụng thực tế 36
5.1 Tổng quan bài toán 36
5.2 Tập dữ liệu Groceries 36
5.3 Các độ đo đánh giá luật kết hợp 36
5.4 Cài đặt và quy trình thực nghiệm 37
5.5 Top-10 luật kết hợp theo Lift 38
5.6 Phân tích và ý nghĩa kinh doanh 39
5.6.1 Nhóm luật về rau củ (Luật 1–8) 39
5.6.2 Luật về sản phẩm từ sữa (Luật 2, 9, 10) 40
5.6.3 Nhận xét tổng hợp 40
5.7 Đề xuất ứng dụng thực tiễn 40

Tài liệu tham khảo 42

Phụ lục 43

1 Nền tảng lý thuyết

1.1 Bài toán frequent itemset mining

1.1.1 Cơ sở dữ liệu giao dịch

Gọi $I = \{i_1, i_2, \ldots, i_m\}$ là tập tất cả các item. Một cơ sở dữ liệu giao dịch được định nghĩa là:

$$D = \{T_1, T_2, \ldots, T_n\}$$

trong đó mỗi giao dịch T_k là một tập con của I:

$$T_k \subseteq I, \quad \forall k = 1, 2, \ldots, n$$

Mỗi giao dịch biểu diễn một lần xuất hiện đồng thời của các item. Trong bài toán frequent itemset mining, mục tiêu là tìm các tập item xuất hiện thường xuyên trong cơ sở dữ liệu giao dịch.

1.1.2 Độ hỗ trợ

Với một itemset $X \subseteq I$, độ hỗ trợ tuyệt đối của X là số lượng giao dịch trong cơ sở dữ liệu chứa X:

$$sup(X) = |\{T \in D \mid X \subseteq T\}|$$

Ngoài support tuyệt đối, ta còn dùng support tương đối, được định nghĩa là tỉ lệ số giao dịch chứa X trên tổng số giao dịch:

$$rsup(X) = \frac{sup(X)}{|D|}$$

Trong đó:

- $sup(X)$ là support tuyệt đối,
- $rsup(X)$ là support tương đối,
- $|D| = n$ là số lượng giao dịch trong cơ sở dữ liệu.

Support tuyệt đối thường được dùng trực tiếp trong quá trình cài đặt và khai phá, trong khi support tương đối thuận tiện hơn khi so sánh giữa các tập dữ liệu có kích thước khác nhau.

1.1.3 Tập phổ biến

Cho một ngưỡng hỗ trợ tối thiểu *minsup*. Một itemset $X \subseteq I$ được gọi là *tập phổ biến* nếu support của nó không nhỏ hơn ngưỡng này:

$$sup(X) \geq minsup$$

Nếu dùng support tương đối thì điều kiện tương đương là:

$$rsup(X) \geq minsup$$

với *minsup* lúc này được hiểu là một tỉ lệ trong khoảng $[0,1]$.

Tập các itemset thỏa điều kiện trên được gọi là tập frequent itemset của cơ sở dữ liệu D.

1.1.4 Tập đóng và tập tối đại

Một frequent itemset X được gọi là *tập đóng* (*closed itemset*) nếu không tồn tại tập cha thực sự Y sao cho:

$$X \subset Y \quad \text{và} \quad sup(X) = sup(Y)$$

Nói cách khác, X là closed nếu không thể thêm item nào vào X mà vẫn giữ nguyên support.

Một frequent itemset X được gọi là *tập tối đại* (*maximal itemset*) nếu không tồn tại frequent itemset nào Y sao cho:

$$X \subset Y$$

Tức là X là maximal nếu không thể mở rộng thêm item nào mà vẫn còn phổ biến.

Quan hệ giữa ba lớp tập này là:

$$\text{frequent} \supseteq \text{closed} \supseteq \text{maximal}$$

Điều này có nghĩa là:

- mọi maximal itemset đều là closed itemset,
- mọi closed itemset đều là frequent itemset,
- nhưng chiều ngược lại không phải lúc nào cũng đúng.

1.1.5 Tính chất Apriori

Tính chất Apriori, hay còn gọi là tính chất đóng xuống (*downward closure*), phát biểu rằng:

> Nếu một itemset là phổ biến, thì mọi tập con của nó cũng đều phổ biến.

Phát biểu hình thức:

$$X \subseteq Y \Rightarrow sup(X) \geq sup(Y)$$

Từ đó suy ra:

$$sup(Y) \geq minsup \Rightarrow sup(X) \geq minsup$$

với mọi $X \subseteq Y$.
Chứng minh ngắn gọn.
Giả sử $X \subseteq Y$. Nếu một giao dịch T chứa Y, tức là:

$$Y \subseteq T$$

thì do $X \subseteq Y$, suy ra:

$$X \subseteq T$$

Nói cách khác, mọi giao dịch chứa Y đều đồng thời chứa X. Vì vậy số lượng giao dịch chứa X không thể nhỏ hơn số lượng giao dịch chứa Y:

$$sup(X) \geq sup(Y)$$

Suy ra nếu Y là frequent thì mọi tập con của Y cũng frequent. Tính chất này là cơ sở lý thuyết quan trọng của hầu hết các thuật toán frequent itemset mining, vì nó cho phép tỉa bớt không gian tìm kiếm rất mạnh.

1.2 Phân tích thuật toán H-Mine

1.2.1 Ý tưởng cốt lõi

H-Mine là một thuật toán khai phá frequent itemset thuộc nhóm *pattern-growth*. Ý tưởng chính của thuật toán là tránh sinh ứng viên theo kiểu Apriori, đồng thời không cần xây dựng projected database vật lý như một số phương pháp tăng trưởng mẫu khác.

Thay vì tạo ra nhiều candidate rồi quét cơ sở dữ liệu để kiểm tra support, H-Mine tổ chức dữ liệu bằng một cấu trúc gọi là *H-struct*. Cấu trúc này lưu các frequent-item projection của giao dịch và liên kết chúng bằng các *hyper-link*. Nhờ đó, khi khai phá một item hoặc một prefix pattern, thuật toán có thể truy cập trực tiếp projected database tương ứng thông qua các queue liên kết, thay vì phải sao chép hoặc xây lại toàn bộ dữ liệu con.

Trực giác hoạt động của H-Mine là như sau:

- đầu tiên xác định tập frequent 1-itemset và sắp xếp chúng thành một thứ tự toàn cục gọi là F-list,
- với mỗi item trong F-list, thuật toán xem item đó là phần tử mở rộng tiếp theo của prefix hiện tại,
- projected database của mỗi prefix được biểu diễn ảo thông qua các hyper-link,
- từ projected database đó, thuật toán đếm các item cục bộ còn phổ biến rồi tiếp tục đệ quy.

Điểm khác biệt quan trọng của H-Mine so với các thuật toán trước nó là:

- so với Apriori, H-Mine không sinh candidate theo từng mức,
- so với FP-Growth, H-Mine không bắt buộc phải xây projected tree vật lý ở mọi bước,
- H-Mine có chi phí bộ nhớ nhỏ và dễ dự đoán hơn trên dữ liệu thưa, vì phần overhead chủ yếu là các header table và các frequent-item projection đã có sẵn trong bộ nhớ.

1.2.2 Cấu trúc dữ liệu chính

Cấu trúc dữ liệu trung tâm của H-Mine là *H-struct*. H-struct gồm hai thành phần chính:

Frequent-item projections
Mỗi giao dịch ban đầu được lọc bỏ các item không phổ biến, sau đó sắp xếp theo F-list. Phần còn lại được gọi là frequent-item projection của giao dịch.

Ví dụ, giả sử F-list là:

$$a \prec c \prec d \prec e \prec g$$

và giao dịch gốc là:

$$T = \{c, d, e, f, g\}$$

nếu f không phổ biến thì frequent-item projection của giao dịch là:

$$[c, d, e, g]$$

Header table và hyper-link

Header table là một bảng chứa:

- item,
- support của item,
- con trỏ tới đầu queue của item đó.

Các projection có cùng item đầu tiên sẽ được nối với nhau thành một queue bằng hyper-link. Nhờ vậy, projected database theo từng item có thể được truy cập trực tiếp.

Ví dụ, giả sử có các projection:

$$[c, d, e, g], \quad [a, c, d, e], \quad [a, d, e, g], \quad [a, c, d]$$

thì:

- queue của a sẽ nối các projection bắt đầu bằng a,
- queue của c sẽ nối các projection bắt đầu bằng c,
- các queue còn lại có thể rỗng ở mức đầu tiên.

Cấu trúc này giúp thuật toán biểu diễn projected database một cách ảo, thay vì tạo bản sao vật lý cho từng nhánh đệ quy.

Minh họa cấu trúc H-Struct

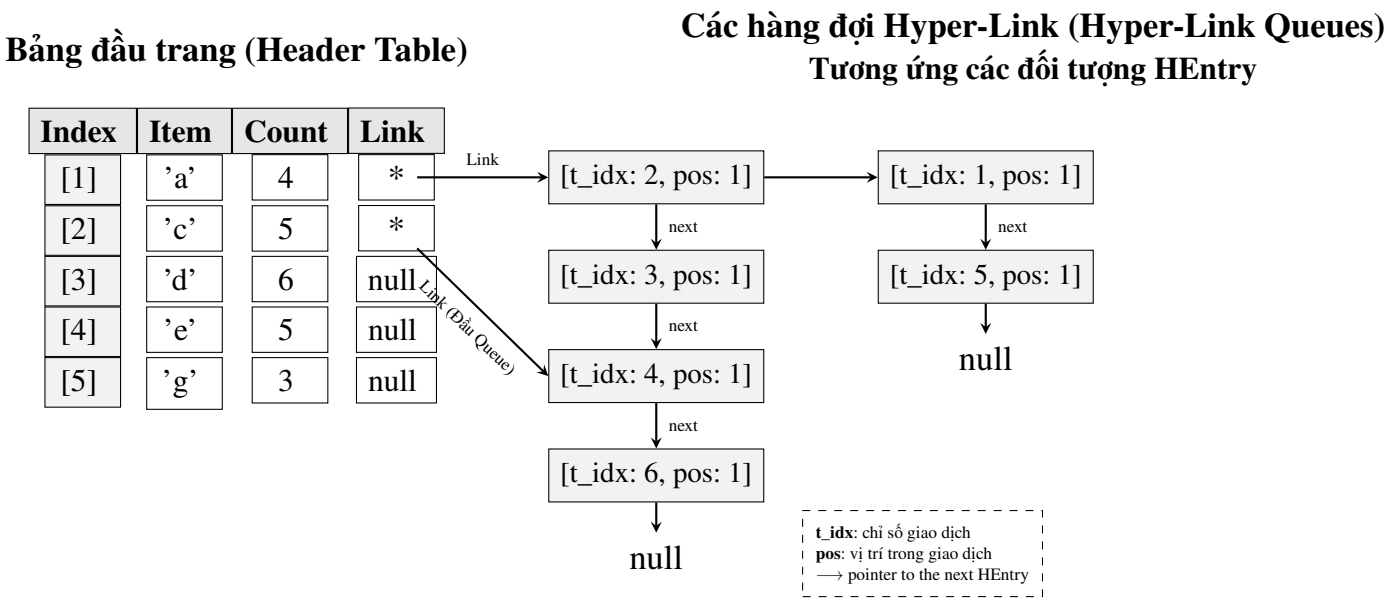

Figure 1.1: Minh họa cấu trúc H-Struct và liên kết Hyper-link

Sơ đồ trên thể hiện cách tổ chức bộ nhớ mà nhóm em đã cài đặt:
HeaderTable:

- **items**: Là mảng **[a, c, d, e, g]**.
- **counts**: Là mảng **[4, 5, 6, 5, 3]**.
- **links**: Là mảng chứa các con trỏ. Trong sơ đồ, tại item 'a', con trỏ trỏ đến danh sách liên kết các giao dịch chứa 'a' ở vị trí đầu tiên (T2, T3, T4, T6). Tại item 'c', con trỏ trỏ đến các giao dịch bắt đầu bằng 'c' (T1, T5).

HEntry (Hyper-link):
Mỗi khối **[t_idx: X, pos: Y]** trên sơ đồ chính là một đối tượng **HEntry**:

- **t_idx**: Chỉ số của giao dịch trong cơ sở dữ liệu (Ví dụ: **t_idx: 2** nghĩa là trỏ vào mảng **database[2]** tức là giao dịch T2).
- **pos**: Vị trí của item hiện tại trong giao dịch đó (Chỉ số Julia bắt đầu từ 1). Trong sơ đồ ban đầu, các item 'a' đều nằm ở vị trí thứ 1 trong mảng của chúng (**pos = 1**).
- **next**: Con trỏ liên kết (mũi tên hướng xuống) nối các giao dịch có chung tiền tố (prefix) lại thành một hàng đợi ảo (Virtual Projection), giúp thuật toán vận hành mà không cần sao chép mảng dữ liệu vật lý nhiều lần.

1.2.3 Mã giả thuật toán

Thuật toán H-Mine gồm hai giai đoạn chính: xây dựng *H-struct* ban đầu và khai phá đệ quy trên các cơ sở dữ liệu chiếu.

1.2.3.1 Thuật toán chính H-Mine

Algorithm 1 Thuật toán H-Mine

Require: Cơ sở dữ liệu giao dịch D, ngưỡng hỗ trợ tối thiểu *minsup*
Ensure: Toàn bộ frequent itemset trong D

```
Quét D lần thứ nhất để đếm support của các 1-itemset
Xác định tập frequent item và sắp xếp chúng thành F-list
Quét D lần thứ hai để xây dựng H-struct:
    - loại bỏ các item không phổ biến trong mỗi giao dịch
    - sắp xếp các item còn lại theo thứ tự trong F-list
    - tạo header table và các hyper-link queue tương ứng
for mỗi item x_i trong F-list do
    Xuất mẫu phổ biến {x_i} cùng support của nó
    Gọi thủ tục đệ quy để khai phá cơ sở dữ liệu chiếu theo x_i
    Điều chỉnh lại các hyper-link để chuẩn bị cho các nhánh kế tiếp
end for
```

1.2.3.2 Thủ tục đệ quy

Algorithm 2 Thủ tục khai phá đệ quy của H-Mine

Require: Tiền tố hiện tại P, header table H, ngưỡng hỗ trợ tối thiểu *minsup*
Ensure: Các frequent itemset mở rộng từ P

```
Duyệt queue tương ứng với tiền tố P
Đếm support cục bộ của các item xuất hiện phía sau P
Xác định các item còn phổ biến trong cơ sở dữ liệu chiếu
Tạo header table cục bộ H_P
Liên kết các projected transaction vào các queue tương ứng trong H_P
for mỗi item phổ biến cục bộ x trong H_P do
    Xuất mẫu mới P ∪ {x} cùng support của nó
    Gọi đệ quy trên tiền tố mới P ∪ {x}
end for
Sau khi xử lý xong một nhánh, điều chỉnh hyper-link để tiếp tục khai phá các nhánh cùng mức
```

1.2.3.3 Giải thích ý nghĩa các bước.

- Bước quét đầu tiên dùng để xác định tập item phổ biến toàn cục.
- Bước quét thứ hai xây H-struct, là nền tảng để các projected database về sau chỉ còn là các queue liên kết.
- Trong mỗi lần đệ quy, thuật toán chỉ xét các item nằm bên phải prefix theo thứ tự F-list, nhờ đó tránh sinh trùng lặp.
- Việc điều chỉnh link sau mỗi nhánh giúp tái sử dụng cấu trúc hiện có, thay vì tạo projected database mới hoàn toàn.

1.2.4 Phân tích độ phức tạp

Trước khi phân tích, ta quy ước các biến số dựa theo thuật toán H-Mine:

- N: Tổng số giao dịch trong CSDL (sau khi lọc).
- L và L_{max}: Độ dài trung bình và cực đại của một giao dịch.
- M: Kích thước cấu trúc F-list (số lượng phần tử phổ biến).
- K: Tổng số lượng mẫu phổ biến (frequent itemsets) được sinh ra.
- V_p: Tần suất (support) của mẫu p (chiều dài của queue chứa mẫu).

1.2.4.1 Độ phức tạp thời gian

Tổng thời gian chạy dựa trên cơ chế duyệt các con trỏ H-link là tổng chi phí sắp xếp dữ liệu gốc cộng với chi phí duyệt các đệ quy: $O\left(N \cdot L \log L + \sum_{p=1}^{K} V_p \cdot L\right)$

- **Trường hợp tốt nhất ($O(N \cdot L \log L)$):** Xảy ra khi *minsup* đạt ngưỡng rất cao, $K \to 0$. Thuật toán sẽ dừng lại ngay sau khi đếm support và sắp xếp CSDL ban đầu, không có bất kỳ mẫu mở rộng nào kích hoạt đệ quy.
- **Trường hợp trung bình:** Phụ thuộc trực tiếp vào phân bố dữ liệu và số lượng K. Thời gian bị chi phối chủ yếu bởi chi phí duyệt danh sách *hyper-link* trong đệ quy: $\sum V_p \cdot L$.
- **Trường hợp xấu nhất ($O(K \cdot N \cdot L)$):** Khi *minsup* cực thấp làm bùng nổ tổ hợp số mẫu K (có thể tiệm cận 2^M). Thuật toán phải duyệt đệ quy ở độ sâu lớn với lượng tham chiếu độ trễ cực cao, gần như toàn bộ CSDL luôn đáp ứng điều kiện $V_p \approx N$.

1.2.4.2 Độ phức tạp không gian

Tối ưu không gian (bộ nhớ) là ưu điểm cốt lõi nhất của H-Mine. Tổng định mức RAM sử dụng là:

$$O(N \cdot L + L_{max} \cdot M)$$

- $O(N \cdot L)$: Không gian cấp phát Mảng dữ liệu 2D Projected ảo lưu trên RAM. H-Mine chỉ khởi tạo khối này 1 lần vĩnh viễn và không bị sao chép sinh bản phụ chéo qua các nhánh đệ quy.
- $O(L_{max} \cdot M)$: Chi phí Overhead tạo ra các *HeaderTable* (chứa Array Links, Counts, Items cục bộ). Khi Call Stack đệ quy đạt đáy tối đa L_{max}, yếu tố này vẫn vô cùng nhỏ bé so sánh với CSDL. Yếu tố này giúp H-Mine vượt trội hoàn toàn về mặt dự đoán cấp phát RAM để tránh Crash Memory.

1.2.4.3 Các yếu tố ảnh hưởng chính đến hiệu năng

Hiệu năng của H-Mine phụ thuộc vào một số yếu tố chính:

- **Kích thước cơ sở dữ liệu:** số giao dịch càng lớn thì chi phí quét và khai phá projected database càng cao.

- **Ngưỡng minsup:** minsup càng thấp thì số frequent itemset càng nhiều, số nhánh đệ quy càng lớn.
- **Độ dài giao dịch trung bình:** giao dịch càng dài thì số item đồng xuất hiện càng nhiều, projected database càng phức tạp.
- **Mức độ thưa hay dày của dữ liệu:** trên dữ liệu thưa, H-struct thường có lợi hơn; trên dữ liệu dày, projected database có thể trở nên thích hợp hơn cho việc chuyển sang FP-tree.
- **Thứ tự item trong F-list:** thứ tự này ảnh hưởng trực tiếp đến cách phân hoạch không gian tìm kiếm và hiệu quả nén cục bộ.

1.2.5 So sánh lịch sử

Trong dòng phát triển của frequent itemset mining, H-Mine xuất hiện sau các thuật toán sinh ứng viên như Apriori và cùng thế hệ với các phương pháp pattern-growth như FP-Growth.

1.2.5.1 Hạn chế của các thuật toán trước:

- **Apriori** khai phá theo từng mức và phải sinh rất nhiều candidate, đặc biệt khi dữ liệu dày hoặc minsup thấp.
- **FP-Growth** loại bỏ candidate generation bằng FP-tree, nhưng trên dữ liệu thưa hoặc rất lớn, việc xây và quản lý các projected tree có thể tốn bộ nhớ và khó dự đoán chi phí.

Đóng góp của H-Mine

H-Mine được đề xuất để giải quyết hai vấn đề chính:

- giảm chi phí bộ nhớ và làm cho nhu cầu bộ nhớ dễ dự đoán hơn,
- vẫn giữ được ưu thế của hướng pattern-growth, tức không sinh candidate như Apriori.

Điểm mạnh của H-Mine là biểu diễn projected database bằng hyper-link, nhờ đó phù hợp đặc biệt tốt với dữ liệu thưa và các tình huống mà FP-tree không nén tốt.

1.2.5.2 Hạn chế còn lại

Dù hiệu quả, H-Mine vẫn có một số hạn chế:

- khi dữ liệu rất dày, việc chỉ dùng H-struct có thể không tận dụng tốt khả năng chia sẻ prefix như FP-tree,
- bài toán frequent itemset mining tự thân vẫn có không gian đầu ra rất lớn, nên H-Mine không thể loại bỏ hoàn toàn hiện tượng bùng nổ số lượng mẫu,
- hiệu năng thực tế vẫn phụ thuộc mạnh vào minsup và đặc trưng dữ liệu.

Vì vậy, trong các nghiên cứu tiếp theo, nhiều thuật toán tiếp tục khai thác sâu hơn các ý tưởng như:

- biểu diễn dọc bằng tidset hoặc diffset,
- khai phá closed itemset hoặc maximal itemset để giảm output,
- kết hợp động nhiều cấu trúc dữ liệu khác nhau tùy vào mật độ projected database.

Tóm lại, H-Mine là một bước phát triển quan trọng trong nhóm thuật toán pattern-growth, đóng vai trò trung gian giữa các phương pháp tránh candidate generation và các phương pháp tối ưu không gian khai phá trên dữ liệu lớn.

2 Ví dụ minh họa tay

2.1 Ví dụ 1: Cơ sở

Để minh họa cách hoạt động của thuật toán H-Mine, nhóm sử dụng một cơ sở dữ liệu đồ chơi gồm 6 giao dịch và 6 item, với ngưỡng hỗ trợ tối thiểu $minsup = 3$.

Table 2.1: Cơ sở dữ liệu nhỏ cho ví dụ cơ sở

TID	Items
T_1	$\{c,d,e,f,g\}$
T_2	$\{a,c,d,e\}$
T_3	$\{a,d,e,g\}$
T_4	$\{a,c,d\}$
T_5	$\{c,d,e,g\}$
T_6	$\{a,c,d,e\}$

2.1.1 Bước 1: đếm support 1-itemset và xây dựng F-list

Đếm số lần xuất hiện của từng item trên toàn bộ CSDL:

$$sup(a) = 4, \quad sup(c) = 5, \quad sup(d) = 6, \quad sup(e) = 5, \quad sup(f) = 1, \quad sup(g) = 3$$

Với $minsup = 3$, item f bị loại. Tập item phổ biến là:

$$L = \{a,c,d,e,g\}$$

Nhóm sử dụng thứ tự tăng dần theo chữ cái để tạo *F-list*:

$$a \prec c \prec d \prec e \prec g$$

2.1.2 Bước 2: tạo frequent-item projection

Mỗi giao dịch được lọc bỏ item không phổ biến và sắp xếp theo F-list:

Table 2.2: Frequent-item projection của từng giao dịch

TID	Giao dịch gốc	Projection theo F-list
T_1	$\{c,d,e,f,g\}$	$[c,d,e,g]$
T_2	$\{a,c,d,e\}$	$[a,c,d,e]$
T_3	$\{a,d,e,g\}$	$[a,d,e,g]$
T_4	$\{a,c,d\}$	$[a,c,d]$
T_5	$\{c,d,e,g\}$	$[c,d,e,g]$
T_6	$\{a,c,d,e\}$	$[a,c,d,e]$

Từ đây, header table ban đầu có dạng:

Item	Support
a	4
c	5
d	6
e	5
g	3

Các queue ban đầu của H-struct:

$$a\text{-queue} = \{T_2, T_3, T_4, T_6\}, \qquad c\text{-queue} = \{T_1, T_5\}$$

còn d-queue, e-queue và g-queue rỗng vì không có projection nào bắt đầu bằng các item đó.

2.1.3 Bước 3: khai phá nhánh chứa item a

Xét a-projected database, gồm các projection:

$$T_2 : [c,d,e], \quad T_3 : [d,e,g], \quad T_4 : [c,d], \quad T_6 : [c,d,e]$$

Đếm support cục bộ của các item đứng bên phải a:

$$sup_a(c) = 3, \quad sup_a(d) = 4, \quad sup_a(e) = 3, \quad sup_a(g) = 1$$

Do đó các mẫu phổ biến sinh ra ngay ở mức này là:

$$\{ac : 3,\ ad : 4,\ ae : 3\}$$

và local F-list của nhánh a là:

$$c \prec d \prec e$$

2.1.3.1 Nhánh *ac* *ac*-projected database gồm các suffix sau c trong các giao dịch có chứa ac:

$$T_2 : [d, e], \quad T_4 : [d], \quad T_6 : [d, e]$$

Đếm support cục bộ:

$$sup_{ac}(d) = 3, \quad sup_{ac}(e) = 2$$

Suy ra chỉ có:

$$acd : 3$$

2.1.3.2 Nhánh *ad* Xét các suffix sau d trong các giao dịch của a-projected database:

$$T_2 : [e], \quad T_3 : [e, g], \quad T_4 : [\,], \quad T_6 : [e]$$

Khi đó:

$$sup_{ad}(e) = 3, \quad sup_{ad}(g) = 1$$

Suy ra:

$$ade : 3$$

2.1.3.3 Nhánh *ae* Với ae-projected database, phía sau e không còn item nào đủ minsup, nên không sinh thêm mẫu mới.

Vậy toàn bộ các frequent itemset chứa a là:

$$\{a : 4,\ ac : 3,\ ad : 4,\ ae : 3,\ acd : 3,\ ade : 3\}$$

2.1.4 Bước 4: khai phá các nhánh còn lại

2.1.4.1 Nhánh c nhưng không chứa a Các giao dịch chứa c nhưng không xét lại các mẫu đã thuộc nhánh a là:

$$T_1 : [d,e,g], \quad T_5 : [d,e,g]$$

sau khi cộng thêm phần điều chỉnh link từ các giao dịch trong a-queue sang queue thích hợp, tập giao dịch hiệu lực cho nhánh c cho ta các support cục bộ:

$$sup_c(d) = 5, \quad sup_c(e) = 4, \quad sup_c(g) = 2$$

Suy ra:

$$cd : 5, \quad ce : 4$$

Xét tiếp nhánh cd, ta có:

$$sup_{cd}(e) = 4, \quad sup_{cd}(g) = 2$$

nên sinh thêm:

$$cde : 4$$

2.1.4.2 Nhánh d nhưng không chứa a,c Sau khi loại các nhánh đã xét trước đó, support cục bộ còn lại của các item bên phải d là:

$$sup_d(e) = 5, \quad sup_d(g) = 3$$

Suy ra:

$$de : 5, \quad dg : 3$$

Từ nhánh de, ta có:

$$sup_{de}(g) = 3$$

nên sinh thêm:

$$deg : 3$$

2.1.4.3 Nhánh e nhưng không chứa a, c, d

Chỉ còn:

$$eg : 3$$

2.1.4.4 Nhánh g

Không còn item nào phía sau g, nên dừng.

2.1.5 Kết quả cuối cùng

Toàn bộ frequent itemset tìm được là:

$$\{a : 4,\ c : 5,\ d : 6,\ e : 5,\ g : 3,\ ac : 3,\ ad : 4,\ ae : 3,$$
$$acd : 3,\ ade : 3,\ cd : 5,\ ce : 4,\ cde : 4,\ de : 5,\ dg : 3,\ deg : 3,\ eg : 3\}$$

2.1.6 Kiểm tra chéo bằng liệt kê thủ công

Liệt kê toàn bộ itemset và giữ lại những itemset có support không nhỏ hơn 3:

- 1-itemset phổ biến:
$$\{a, c, d, e, g\}$$
với support tương ứng:
$$\{4, 5, 6, 5, 3\}$$
- 2-itemset phổ biến:
$$ac : 3,\ ad : 4,\ ae : 3,\ cd : 5,\ ce : 4,\ de : 5,\ dg : 3,\ eg : 3$$
- 3-itemset phổ biến:
$$acd : 3,\ ade : 3,\ cde : 4,\ deg : 3$$
- Không có 4-itemset phổ biến vì:
$$sup(acde) = 2, \quad sup(cdeg) = 2$$

Kết quả liệt kê thủ công hoàn toàn trùng với kết quả khai phá bằng từng bước của H-Mine. Điều này xác nhận ví dụ tay là nhất quán và đúng.

2.2 Ví dụ 2: Tình huống đặc biệt

Trong H-Mine, một tình huống đặc biệt quan trọng là khi cơ sở dữ liệu hoặc projected database có mức chia sẻ prefix rất cao, tức nhiều giao dịch gần như cùng đi theo một nhánh chính. Khi đó, projected database trở nên *dày đặc* và có cấu trúc gần *single path*. Đây là trường hợp đáng chú ý vì H-struct vẫn xử lý đúng, nhưng nếu projected database đủ đậm đặc thì việc chuyển sang FP-tree sẽ có lợi hơn nhờ khả năng nén prefix chung.

Để minh họa, xét cơ sở dữ liệu sau với 5 giao dịch và 4 item, chọn:

$$minsup = 2$$

Table 2.3: Cơ sở dữ liệu cho ví dụ tình huống đặc biệt

TID	**Items**
T_1	$\{a,b,c,d\}$
T_2	$\{a,b,c\}$
T_3	$\{a,b\}$
T_4	$\{a,b,c,d\}$
T_5	$\{a,b,c\}$

2.2.1 Bước 1: đếm support 1-itemset và xây dựng F-list

Đếm support của từng item trên toàn bộ cơ sở dữ liệu:

$$sup(a) = 5, \quad sup(b) = 5, \quad sup(c) = 4, \quad sup(d) = 2$$

Vì tất cả đều có support không nhỏ hơn minsup, nên toàn bộ các item đều phổ biến. Nhóm chọn thứ tự F-list tăng dần theo chữ cái:

$$a \prec b \prec c \prec d$$

2.2.2 Bước 2: tạo frequent-item projection

Do tất cả các item đều phổ biến, nên sau khi lọc theo minsup, các projection giữ nguyên và chỉ cần sắp theo F-list:

Table 2.4: Frequent-item projection của ví dụ đặc biệt

TID	Projection theo F-list
T_1	$[a,b,c,d]$
T_2	$[a,b,c]$
T_3	$[a,b]$
T_4	$[a,b,c,d]$
T_5	$[a,b,c]$

Header table ban đầu là:

Item	Support
a	5
b	5
c	4
d	2

Có thể thấy ngay rằng mọi giao dịch đều bắt đầu bằng a, sau đó tiếp tục gần như đồng nhất với prefix $b \to c$. Đây chính là dấu hiệu của trường hợp projected database rất đậm đặc.

2.2.3 Bước 3: khai phá nhánh chứa item a

Xét a-projected database, tức lấy phần suffix phía sau a trong mỗi giao dịch:

$$T_1 : [b,c,d], \quad T_2 : [b,c], \quad T_3 : [b], \quad T_4 : [b,c,d], \quad T_5 : [b,c]$$

Đếm support cục bộ của các item đứng bên phải a:

$$sup_a(b) = 5, \quad sup_a(c) = 4, \quad sup_a(d) = 2$$

Do đó các frequent itemset sinh ra ngay ở mức này là:

$$ab : 5, \quad ac : 4, \quad ad : 2$$

và local F-list của nhánh a là:

$$b \prec c \prec d$$

2.2.3.1 Nhánh *ab* Xét các suffix sau *b* trong *a*-projected database:

$$T_1 : [c,d], \quad T_2 : [c], \quad T_3 : [\,], \quad T_4 : [c,d], \quad T_5 : [c]$$

Đếm support cục bộ:

$$sup_{ab}(c) = 4, \quad sup_{ab}(d) = 2$$

Suy ra:

$$abc : 4, \quad abd : 2$$

2.2.3.2 Nhánh *abc* Xét tiếp suffix sau *c* trong các giao dịch chứa *abc*:

$$T_1 : [d], \quad T_2 : [\,], \quad T_4 : [d], \quad T_5 : [\,]$$

Khi đó:

$$sup_{abc}(d) = 2$$

nên sinh thêm:

$$abcd : 2$$

2.2.3.3 Nhánh *ac* Quay lại nhánh *a*, xét trực tiếp nhánh *ac*. Các suffix sau *c* trong các giao dịch chứa *ac* là:

$$T_1 : [d], \quad T_2 : [\,], \quad T_4 : [d], \quad T_5 : [\,]$$

Suy ra:

$$sup_{ac}(d) = 2$$

nên sinh thêm:

$$acd : 2$$

2.2.3.4 Nhánh ad Sau d không còn item nào nữa, nên nhánh ad dừng.

Vậy toàn bộ frequent itemset chứa a là:

$$\{a:5,\ ab:5,\ ac:4,\ ad:2,\ abc:4,\ abd:2,\ acd:2,\ abcd:2\}$$

2.2.4 Bước 4: khai phá các nhánh còn lại

2.2.4.1 Nhánh b nhưng không chứa a Do mọi giao dịch đều đã đi qua a, khi xét nhánh b ở mức gốc thì các giao dịch hiệu lực tương ứng cho ra:

$$sup_b(c) = 4, \quad sup_b(d) = 2$$

nên sinh:

$$bc:4, \quad bd:2$$

Xét tiếp nhánh bc, ta có:

$$sup_{bc}(d) = 2$$

nên sinh thêm:

$$bcd:2$$

2.2.4.2 Nhánh c nhưng không chứa a,b Từ dữ liệu gốc, support của c là:

$$c:4$$

và khi xét phía sau c ta có:

$$sup_c(d) = 2$$

nên sinh:

$$cd:2$$

2.2.4.3 Nhánh d Không còn item nào phía sau d, nên dừng.

2.2.5 Kết quả cuối cùng

Toàn bộ frequent itemset thu được là:

$$\{a:5,\ b:5,\ c:4,\ d:2,\ ab:5,\ ac:4,\ ad:2,\ bc:4,\ bd:2,\ cd:2,$$
$$abc:4,\ abd:2,\ acd:2,\ bcd:2,\ abcd:2\}$$

2.2.6 Kiểm tra chéo bằng liệt kê thủ công

Liệt kê toàn bộ các itemset có support không nhỏ hơn 2:

- 1-itemset phổ biến:
$$a:5, \quad b:5, \quad c:4, \quad d:2$$
- 2-itemset phổ biến:
$$ab:5, \quad ac:4, \quad ad:2, \quad bc:4, \quad bd:2, \quad cd:2$$
- 3-itemset phổ biến:
$$abc:4, \quad abd:2, \quad acd:2, \quad bcd:2$$
- 4-itemset phổ biến:
$$abcd:2$$

Kết quả liệt kê thủ công hoàn toàn trùng với kết quả khai phá từng bước ở trên.

2.2.7 Vì sao đây là tình huống đặc biệt quan trọng?

Điểm đặc biệt của ví dụ này nằm ở chỗ các giao dịch chia sẻ prefix chung rất dài:

$$[a,b,c,d],\ [a,b,c],\ [a,b],\ [a,b,c,d],\ [a,b,c]$$

Hầu như toàn bộ cơ sở dữ liệu đều xoay quanh nhánh chính:

$$a \to b \to c \to d$$

Điều này dẫn đến hai hệ quả quan trọng:

- Nếu tiếp tục khai phá hoàn toàn bằng H-struct, thuật toán vẫn đúng nhưng sẽ phải duyệt nhiều projected queue có nội dung rất giống nhau.
- Nếu chuyển projected database này sang FP-tree, toàn bộ dữ liệu có thể được nén thành một cấu trúc gần như single-path, từ đó giảm chi phí lưu trữ và khai phá.

Nói cách khác, đây là trường hợp tiêu biểu cho thấy H-Mine mạnh ở dữ liệu thưa, nhưng khi projected database trở nên rất dày và chia sẻ prefix mạnh thì việc chuyển sang FP-tree là hợp lý. Đây cũng chính là động cơ cho hướng tối ưu hóa hybrid mà nhóm sử dụng trong phần cài đặt.

3 Cài đặt

3.1 Môi trường và công cụ

Nhóm cài đặt thuật toán bằng **Julia**, phù hợp với định hướng ưu tiên của đề bài. Môi trường thực nghiệm và phát triển gồm:

- Ngôn ngữ lập trình: Julia
- Môi trường chạy mã nguồn: Visual Studio Code
- Công cụ đối chiếu kết quả tham chiếu: SPMF (Java)
- Định dạng dữ liệu vào/ra: chuẩn SPMF, mỗi dòng là một giao dịch, các item cách nhau bởi dấu cách

Lý do chọn Julia là ngôn ngữ này có hiệu năng cao, hỗ trợ tốt cho tính toán số, cho phép kiểm soát bộ nhớ tốt hơn Python trong các vòng lặp nóng (*hot loop*), đồng thời vẫn giữ được cú pháp ngắn gọn để hiện thực thuật toán từ bài báo.

3.2 Cấu trúc dữ liệu chính

Thuật toán sử dụng hai cấu trúc dữ liệu cốt lõi:

3.2.1 HEntry

Mỗi node của hyper-link queue được biểu diễn bằng cấu trúc:

- `t_idx`: chỉ số giao dịch trong cơ sở dữ liệu đã lọc

- `pos`: vị trí item hiện tại trong giao dịch
- `next`: con trỏ tới phần tử tiếp theo trong queue

Cấu trúc này cho phép mô phỏng projected database một cách *ảo*, không cần sao chép giao dịch vật lý.

3.2.2 HeaderTable

Mỗi bảng header gồm ba vector song song:

- `items`: danh sách item phổ biến theo F-list
- `counts`: support tương ứng
- `links`: đầu queue hyper-link của từng item

Nhờ đó, mỗi mức đệ quy chỉ cần tạo một header table cục bộ thay vì một projected database đầy đủ.

3.3 Quy trình cài đặt bản cơ bản

Bản cơ bản được hiện thực theo đúng tinh thần của H-mine(Mem) trong bài báo, gồm các bước chính sau:

1. Đọc toàn bộ CSDL giao dịch từ file SPMF.
2. Quét CSDL để đếm support 1-itemset và xác định *F-list*.
3. Lọc từng giao dịch, loại bỏ item không phổ biến và sắp xếp lại theo thứ tự F-list.
4. Xây dựng *HeaderTable* ban đầu và các queue hyper-link tương ứng.
5. Gọi đệ quy hàm `mine_h(...)` để:
    - xuất pattern hiện tại,
    - đếm support cục bộ của các item phía sau,
    - tạo local header,
    - xây virtual projection,
    - đệ quy trên projected database cục bộ,
    - điều chỉnh lại hyper-link sau khi hoàn tất từng nhánh.

Đặc điểm của bản cơ bản là bám sát logic trong paper, dễ kiểm chứng tính đúng đắn và thuận tiện để đối chiếu với bản tối ưu sau này.

3.4 Quy trình cài đặt bản tối ưu

Bên cạnh bản cơ bản, nhóm xây dựng thêm bản tối ưu với hai hướng cải tiến chính.

3.4.1 Tối ưu truy cập và cấp phát bộ nhớ

Bản tối ưu thay thế nhiều thao tác tra cứu tốn chi phí bằng mảng ánh xạ trực tiếp từ item sang chỉ số:

`item2idx[item]` → vị trí của item trong header

Điều này giúp tránh sử dụng `Dict` trong các vòng lặp nóng. Ngoài ra, các mảng như `local_counts` và `parent_idx_to_local` được cấp phát một lần bên ngoài vòng lặp và tái sử dụng bằng `fill!`, giúp giảm số lần cấp phát bộ nhớ động.

3.4.2 Tích hợp cơ chế chuyển sang FP-tree trên dữ liệu dày

Trong `hmine_optimized.jl`, nhóm bổ sung phân tích mật độ cục bộ của projected database. Nếu dữ liệu tại nhánh hiện tại đủ dày đặc, thuật toán sẽ xây một FP-tree cục bộ và khai phá bằng hàm `mine_fp_tree(...)`.

Ý tưởng này bám theo hướng tích hợp động H-struct và FP-tree trong paper, nhằm khai thác ưu thế của H-struct trên dữ liệu thưa và ưu thế của FP-tree trên projected database dày.

3.5 Đầu vào và đầu ra

3.5.1 Đọc dữ liệu

Hàm `read_spmf(...)` đọc file văn bản theo chuẩn SPMF, trong đó mỗi dòng là một giao dịch, các item là số nguyên và được ngăn cách bằng dấu cách. Các dòng rỗng hoặc dòng chú thích bắt đầu bởi ký tự `@` được bỏ qua.

3.5.2 Ghi kết quả

Mỗi frequent itemset được ghi ra file theo định dạng:

```
item1 item2 ... #SUP: support
```

Định dạng này tương thích với đầu ra của SPMF và giúp việc đối chiếu kết quả thuận tiện.

3.6 Kiểm thử tính đúng đắn

Nhóm xây dựng bộ kiểm thử tự động trong file `tests/test_correctness.jl` và gom test bằng file `tests/runtests.jl`. Bộ test đối chiếu kết quả của chương trình với SPMF trên 5 cơ sở dữ liệu khác nhau, bao gồm cơ sở dữ liệu đồ chơi ở Chương 2 và 4 tập benchmark chuẩn. Việc đối chiếu được thực hiện trên cả itemset và support của từng itemset.. Với mỗi tập dữ liệu kiểm thử (Cơ sở dữ liệu đồ chơi dùng trong mã nguồn được mã hóa bằng số nguyên để thuận tiện cho việc xử lý. Cụ thể, các item trong ví dụ tay ở Chương 2 được ánh xạ theo quy ước:

$$a \mapsto 1, \quad c \mapsto 3, \quad d \mapsto 4, \quad e \mapsto 5, \quad f \mapsto 6, \quad g \mapsto 7$$

Do đó, file `data/toy/toy_example_ch2.txt` là phiên bản số hóa của ví dụ minh họa tay tương ứng.
Với mỗi tập dữ liệu kiểm thử:

1. Gọi SPMF để sinh file kết quả chuẩn.
2. Chạy bản cơ bản của nhóm.
3. Chạy bản tối ưu của nhóm.
4. Chuẩn hóa từng dòng kết quả về cùng định dạng itemset + support.
5. So sánh tập kết quả của hai bản cài đặt với tập kết quả tham chiếu từ SPMF.

Nếu cả hai tập kết quả trùng hoàn toàn với SPMF thì test được xem là đạt.

4 Thực nghiệm và đánh giá

4.1 Tập dữ liệu benchmark

Nhóm thực nghiệm trên bốn tập dữ liệu benchmark phổ biến trong cộng đồng Frequent Itemset Mining:

- `mushrooms.txt`: tập dữ liệu dày, nhiều item và có nhiều mẫu phổ biến dài

- `retail.txt`: tập dữ liệu thưa, mang tính thực tế của bài toán bán lẻ
- `T10I4D100K.txt`: tập tổng hợp, tương đối thưa
- `accidents.txt`: tập lớn, dày đặc và có chi phí khai phá cao

Bốn tập dữ liệu này đại diện cho nhiều kiểu đặc trưng khác nhau, từ sparse đến dense, từ quy mô vừa đến rất lớn. Vì vậy, chúng phù hợp để đánh giá độ đúng và hiệu năng của H-Mine trên nhiều bối cảnh dữ liệu khác nhau.

4.2 Thiết lập thực nghiệm

Với mỗi tập dữ liệu, nhóm chọn ít nhất 5 giá trị minsup theo thứ tự giảm dần để đo thời gian chạy. Các mức minsup không ép giống nhau hoàn toàn giữa các dataset, mà được chọn sao cho phù hợp với mật độ và kích thước của từng tập dữ liệu. Cách làm này giúp đồ thị phản ánh rõ hơn xu hướng thực tế thay vì rơi vào các vùng quá dễ hoặc quá khó.

Ba phiên bản được so sánh gồm:

- SPMF
- H-Mine bản cơ bản
- H-Mine bản tối ưu

Ngoài thời gian chạy, nhóm còn đo:

- độ đúng đắn của kết quả,
- số lượng frequent itemset sinh ra,
- mức sử dụng bộ nhớ cực đại của bản cơ bản và bản tối ưu,
- khả năng mở rộng theo kích thước dữ liệu trên tập `accidents`,
- ảnh hưởng của độ dài giao dịch trung bình trên dữ liệu tổng hợp.

Đối với thực nghiệm dữ liệu tổng hợp ở mục ảnh hưởng độ dài giao dịch trung bình, nhóm sử dụng seed cố định để bảo đảm khả năng tái sản xuất kết quả khi chạy lại.

4.3 Kiểm tra tính đúng đắn

Hình 4.1 trình bày độ đúng của bản cơ bản và bản tối ưu trên bốn tập benchmark. Có thể thấy trên cả bốn tập dữ liệu, hai đường biểu diễn của bản cơ bản và bản tối ưu đều trùng nhau hoàn toàn tại mức 100% ở mọi giá trị minsup đã khảo sát.

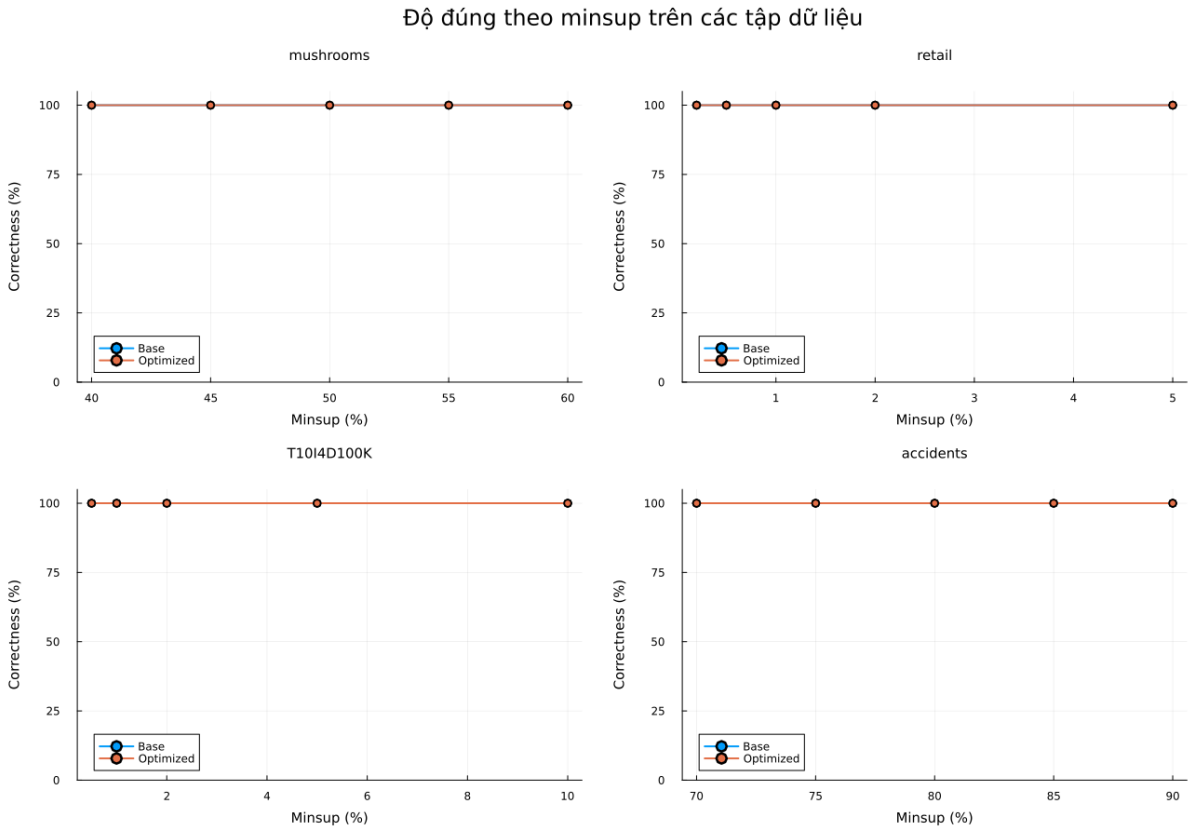

Figure 4.1: Độ đúng theo minsup trên các tập dữ liệu benchmark

Cụ thể:

- Trên `mushrooms`, cả hai bản đều đạt 100% tại các mức minsup $40\%, 45\%, 50\%, 55\%, 60\%$.
- Trên `retail`, cả hai bản đều đạt 100% tại các mức minsup $0.2\%, 0.5\%, 1\%, 2\%, 5\%$.
- Trên `T10I4D100K`, cả hai bản đều đạt 100% tại các mức minsup $0.5\%, 1\%, 2\%, 5\%, 10\%$.
- Trên `accidents`, cả hai bản đều đạt 100% tại các mức minsup $70\%, 75\%, 80\%, 85\%, 90\%$.

Kết quả này cho thấy không chỉ số lượng frequent itemset là đúng, mà từng itemset và support tương ứng cũng khớp hoàn toàn với SPMF. Nhờ đó, các so sánh về thời gian chạy

và bộ nhớ ở các mục tiếp theo có thể được xem là công bằng, vì các phiên bản đều giải cùng một bài toán và cho cùng đầu ra.

4.4 Thời gian chạy theo minsup

Hình 4.2 trình bày thời gian chạy theo minsup trên bốn tập dữ liệu benchmark.

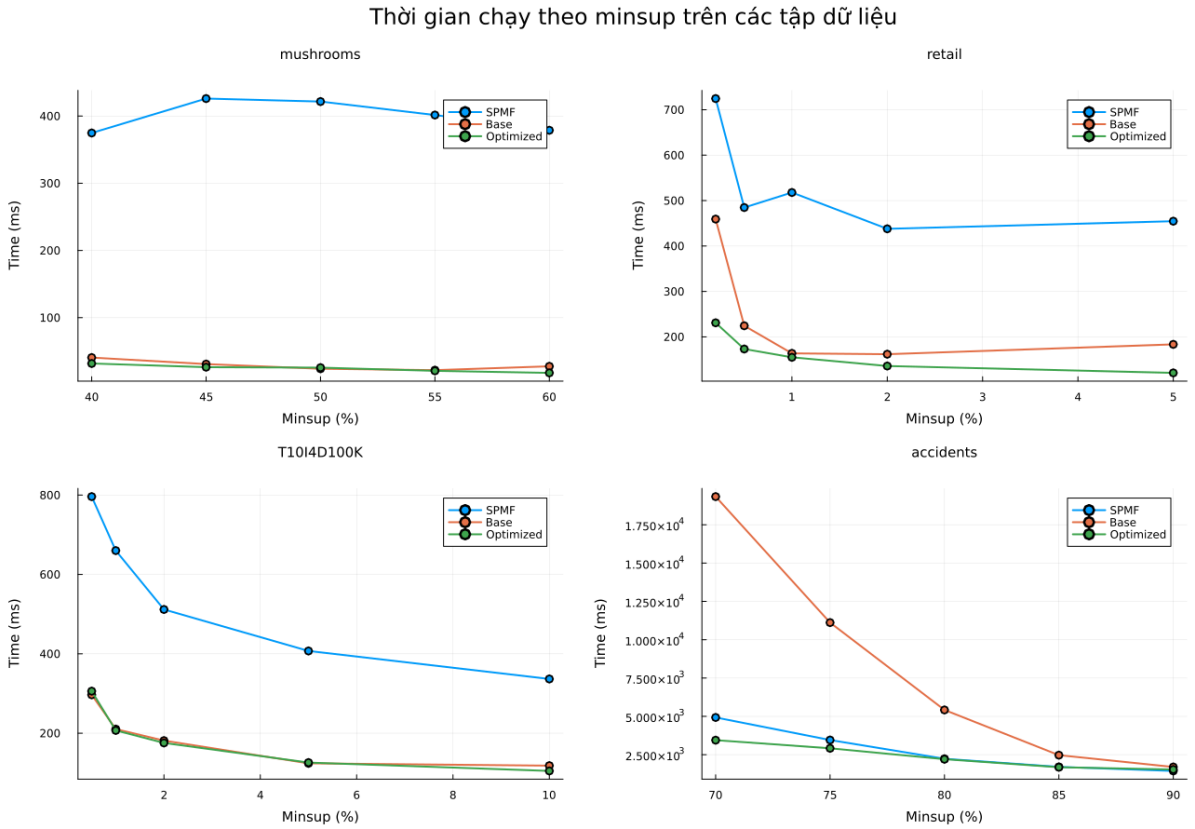

Figure 4.2: Thời gian chạy theo minsup trên bốn tập dữ liệu benchmark

Từ biểu đồ có thể rút ra một số nhận xét cụ thể như sau:

- Trên `mushrooms`, SPMF dao động quanh mức 375–426 ms, trong khi bản cơ bản chỉ khoảng 22–41 ms và bản tối ưu khoảng 18–32 ms. Như vậy, cả hai bản cài đặt Julia đều nhanh hơn SPMF rất rõ rệt; ở minsup 60%, bản tối ưu chỉ mất khoảng 17.83 ms so với 378.81 ms của SPMF.
- Trên `retail`, SPMF nằm trong khoảng 437.87–724.59 ms. Bản cơ bản giảm xuống khoảng 161.81–459.25 ms, còn bản tối ưu khoảng 120.69–230.93 ms. Ở minsup thấp nhất 0.2%, bản tối ưu chỉ mất khoảng 230.93 ms, nhanh hơn đáng kể so với bản cơ bản 459.25 ms và SPMF 724.59 ms.

- Trên `T10I4D100K`, SPMF giảm từ khoảng 796.08 ms ở minsup 0.5% xuống 336.65 ms ở minsup 10%. Bản cơ bản và bản tối ưu đều thấp hơn nhiều, chủ yếu nằm trong khoảng 104–306 ms. Tuy nhiên, chênh lệch giữa hai bản của nhóm không quá lớn; chẳng hạn ở minsup 0.5%, bản tối ưu mất khoảng 305.93 ms, hơi chậm hơn bản cơ bản 296.73 ms.

- Trên `accidents`, đây là tập dữ liệu khó nhất trong bốn tập benchmark. Ở minsup 70%, bản cơ bản mất khoảng 19335.72 ms, trong khi bản tối ưu còn 3446.79 ms và SPMF là 4928.31 ms. Như vậy, bản tối ưu nhanh hơn bản cơ bản rất mạnh và còn vượt SPMF ở cấu hình này. Tương tự, ở minsup 75%, bản tối ưu mất khoảng 2916.44 ms so với 11117.71 ms của bản cơ bản và 3455.67 ms của SPMF.

Nhìn chung, khi minsup giảm thì thời gian chạy tăng lên, vì số lượng frequent itemset sinh ra lớn hơn, projected database sâu hơn và số nhánh đệ quy cũng nhiều hơn. Xu hướng này đặc biệt rõ trên `retail` và `accidents`, nơi số lượng mẫu tăng nhanh khi minsup giảm.

4.5 Số lượng frequent itemset theo minsup

Hình 4.3 minh họa số lượng frequent itemset sinh ra theo minsup trên bốn tập benchmark.

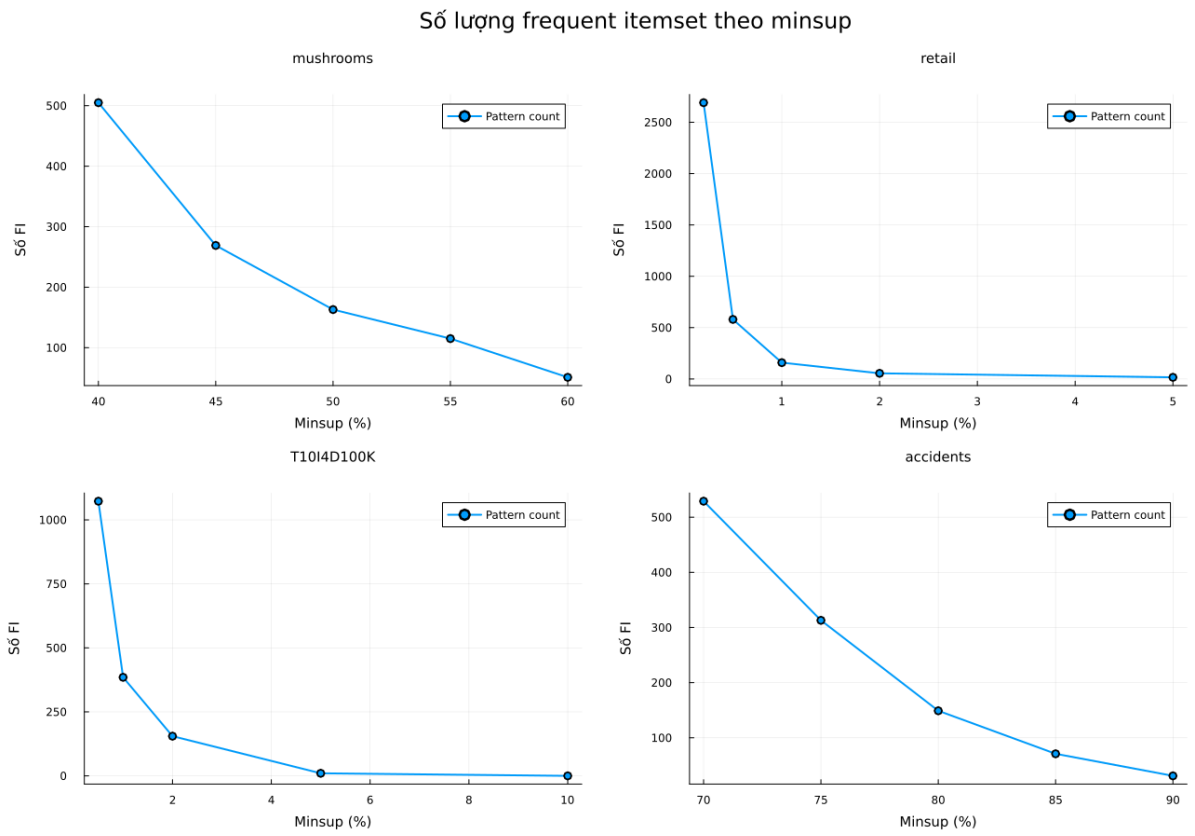

Figure 4.3: Số lượng frequent itemset theo minsup trên các tập dữ liệu benchmark

Kết quả cho thấy số lượng frequent itemset tăng rất rõ khi minsup giảm, nhưng tốc độ tăng khác nhau giữa các dataset:

- Trên `mushrooms`, số frequent itemset tăng từ 51 ở minsup 60% lên 505 ở minsup 40%, tức tăng gần 10 lần.
- Trên `retail`, số frequent itemset tăng mạnh nhất trong bốn tập: từ chỉ 16 ở minsup 5% lên 2691 ở minsup 0.2%, tức tăng hơn 168 lần.
- Trên `T10I4D100K`, số lượng mẫu tăng từ 0 ở minsup 10% lên 1073 ở minsup 0.5%. Điều này cho thấy với minsup cao, gần như không có mẫu phổ biến đáng kể, nhưng khi giảm minsup thì không gian đầu ra mở rộng rất nhanh.
- Trên `accidents`, số frequent itemset tăng từ 31 ở minsup 90% lên 529 ở minsup 70%, tức tăng khoảng 17 lần.

Do ba phiên bản cho kết quả đúng như nhau, số lượng frequent itemset sinh ra giữa SPMF, bản cơ bản và bản tối ưu là trùng khít tại cùng một mức minsup. Vì vậy, trên biểu đồ này chỉ cần thể hiện một đường là đủ để phản ánh xu hướng thay đổi của output size.

Một điểm đáng chú ý là `retail` và `T10I4D100K` là các tập tương đối thưa, nhưng khi minsup giảm sâu thì số lượng mẫu vẫn bùng nổ rất mạnh. Điều này cho thấy output size không chỉ phụ thuộc vào việc dữ liệu dense hay sparse, mà còn phụ thuộc mạnh vào ngưỡng minsup được chọn.

4.6 Sử dụng bộ nhớ

Hình 4.4 trình bày mức bộ nhớ cực đại của bản cơ bản và bản tối ưu trên bốn tập benchmark.

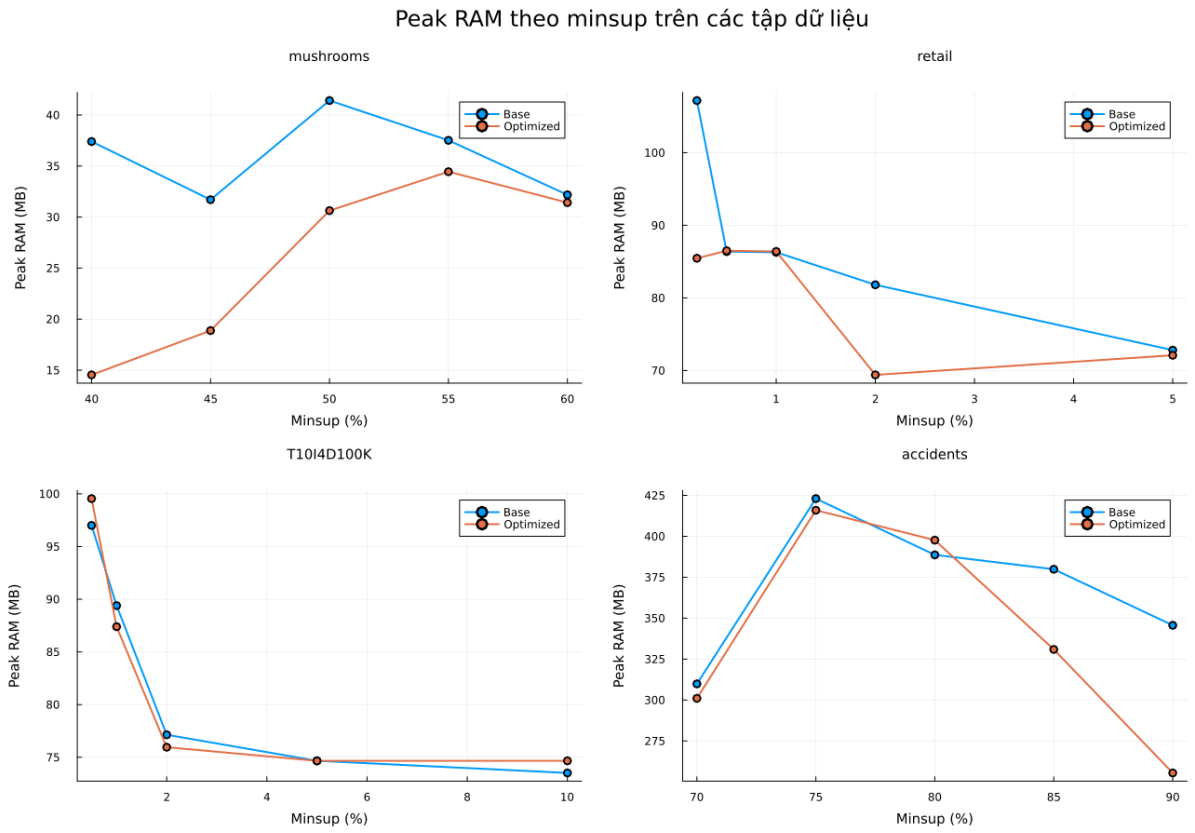

Figure 4.4: Peak RAM theo minsup trên các tập dữ liệu benchmark

Từ biểu đồ, có thể nhận xét cụ thể như sau:

- Trên `mushrooms`, bản tối ưu luôn dùng ít RAM hơn bản cơ bản ở mọi mức minsup. Ví dụ, tại minsup 40%, bản cơ bản dùng khoảng 37.40 MB trong khi bản tối ưu chỉ khoảng 14.54 MB; tại minsup 50%, hai giá trị tương ứng là 41.42 MB và 30.63 MB.
- Trên `retail`, bản tối ưu nhìn chung cũng tiết kiệm bộ nhớ hơn. Ở minsup 0.2%,

bản cơ bản dùng khoảng 107.18 MB còn bản tối ưu là 85.46 MB. Tuy nhiên ở các mức như 0.5% và 1%, chênh lệch giữa hai bản là rất nhỏ, gần như tương đương.

- Trên `T10I4D100K`, mức sử dụng RAM giữa hai bản khá gần nhau. Ở minsup 0.5%, bản tối ưu dùng khoảng 99.55 MB, hơi cao hơn bản cơ bản 97.00 MB; ở minsup 10%, bản tối ưu khoảng 74.66 MB, gần sát bản cơ bản 73.51 MB.
- Trên `accidents`, xu hướng không hoàn toàn ổn định. Ở minsup 90%, bản tối ưu dùng khoảng 255.47 MB, thấp hơn khá nhiều so với 345.64 MB của bản cơ bản. Tuy nhiên, ở minsup 80%, bản tối ưu lại cao hơn nhẹ (397.65 MB so với 388.65 MB). Sau đó, ở minsup 70%, bản tối ưu tiếp tục thấp hơn (301.03 MB so với 309.87 MB).

Như vậy, bản tối ưu thường có lợi thế về bộ nhớ, đặc biệt trên `mushrooms`, `retail` và một số cấu hình của `accidents`. Tuy nhiên, lợi ích này không hoàn toàn đồng đều trên mọi dataset, cho thấy tác động của tối ưu hóa còn phụ thuộc vào cấu trúc projected database và đặc trưng dense/sparse của dữ liệu.

Cần lưu ý rằng đại lượng *peak RAM* ở đây được đo bằng mức bộ nhớ heap nội bộ của Julia trong quá trình chạy, chứ không phải toàn bộ bộ nhớ tiến trình ở cấp hệ điều hành. Vì vậy, kết quả này phù hợp hơn cho việc so sánh tương đối giữa bản cơ bản và bản tối ưu của nhóm.

4.7 Khả năng mở rộng

Nhóm chọn tập `accidents` để đánh giá scalability. Dữ liệu được cắt còn 10%, 25%, 50%, 75% và 100% số giao dịch, sau đó chạy bản tối ưu với cùng một mức minsup cố định. Kết quả về thời gian và bộ nhớ được trình bày trong Hình 4.5.

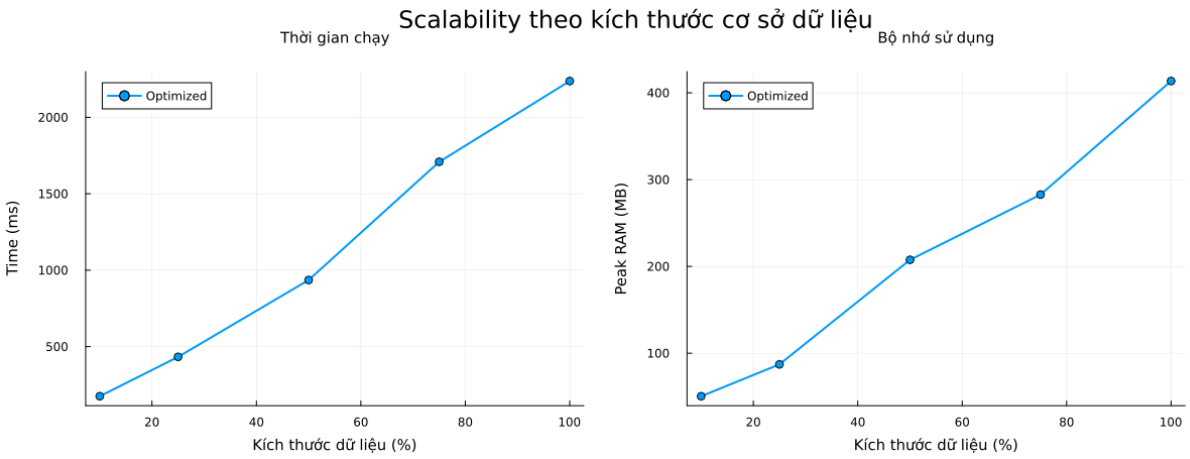

Figure 4.5: Khả năng mở rộng theo kích thước cơ sở dữ liệu trên tập accidents

Từ biểu đồ có thể thấy thời gian chạy tăng khá đều theo kích thước dữ liệu:

- 10%: khoảng 159 ms
- 25%: khoảng 395 ms
- 50%: khoảng 958 ms
- 75%: khoảng 1658 ms
- 100%: khoảng 2179 ms

Từ 10% lên 100%, kích thước dữ liệu tăng 10 lần còn thời gian chạy tăng khoảng 13.7 lần. Điều này cho thấy xu hướng tăng là gần tuyến tính nhưng có độ cong đi lên nhẹ, tức là hơi siêu tuyến tính khi dữ liệu trở nên rất lớn.

Về bộ nhớ, mức sử dụng cũng tăng khá đều:

- 10%: khoảng 51.76 MB
- 25%: khoảng 98.22 MB
- 50%: khoảng 199.00 MB
- 75%: khoảng 317.57 MB
- 100%: khoảng 384.41 MB

Kết quả này cho thấy cài đặt của nhóm giữ được khả năng mở rộng khá tốt trên dữ liệu lớn, dù cả thời gian và bộ nhớ đều tăng rõ rệt theo quy mô đầu vào.

4.8 Ảnh hưởng của độ dài giao dịch trung bình

Để khảo sát ảnh hưởng của độ dài giao dịch trung bình, nhóm tạo một họ cơ sở dữ liệu tổng hợp có cùng số giao dịch nhưng tăng dần số item trung bình trên mỗi giao dịch. Kết quả về thời gian, số lượng frequent itemset và bộ nhớ được trình bày trong Hình 4.6.

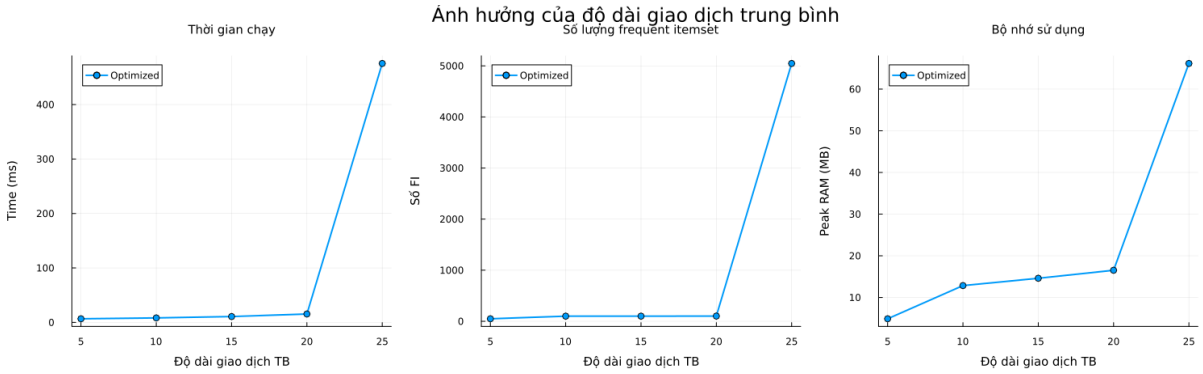

Figure 4.6: Ảnh hưởng của độ dài giao dịch trung bình đến hiệu năng của thuật toán

Biểu đồ cho thấy ảnh hưởng của độ dài giao dịch trung bình là rất rõ, đặc biệt khi độ dài tăng lên mức cao:

- Thời gian chạy tăng từ khoảng 5–15 ms ở các mức độ dài trung bình 5–20, nhưng tăng vọt lên khoảng 470 ms khi độ dài giao dịch trung bình đạt 25.
- Số lượng frequent itemset tăng từ vài chục hoặc khoảng hơn 100 mẫu ở các mức 5–20, nhưng tăng mạnh lên khoảng 5000 mẫu ở mức 25.
- Bộ nhớ sử dụng tăng từ khoảng 5 MB ở độ dài trung bình 5 lên khoảng 16–17 MB ở mức 20, sau đó tăng mạnh lên khoảng 66 MB ở mức 25.

Kết quả này cho thấy khi giao dịch trở nên dài hơn, số item đồng xuất hiện trong mỗi giao dịch tăng lên, kéo theo không gian tìm kiếm lớn hơn rất nhiều. Ở vùng từ 5 đến 20, chi phí tăng còn tương đối nhẹ; nhưng khi đạt đến độ dài trung bình 25, số lượng mẫu và chi phí xử lý tăng đột biến. Điều này phù hợp với lý thuyết của H-Mine: projected database trở nên dày đặc hơn, việc đếm support cục bộ và mở rộng nhánh trở nên tốn kém hơn đáng kể.

4.9 Phân tích kết quả tổng hợp

Từ toàn bộ thực nghiệm, có thể rút ra các nhận xét chính sau:

- **Về tính đúng đắn:** cả hai bản cài đặt đều cho kết quả trùng hoàn toàn với SPMF trên tất cả các cấu hình đã thử, bao gồm cả itemset và support.
- **Về tốc độ:** bản tối ưu mang lại cải thiện rất rõ trên các tập dữ liệu lớn hoặc các cấu hình khó, đặc biệt là `accidents` và `retail`. Chẳng hạn, trên `accidents` ở minsup

70%, thời gian chạy giảm từ khoảng 19335.72 ms của bản cơ bản xuống còn 3446.79 ms ở bản tối ưu.

- **Về bộ nhớ:** bản tối ưu thường dùng ít RAM hơn bản cơ bản, nhưng mức cải thiện không đồng đều giữa các dataset. Trên `mushrooms`, lợi ích là rất rõ; trong khi trên `T10I4D100K`, hai bản gần như tương đương.
- **Về ảnh hưởng của minsup và output size:** khi minsup giảm, số lượng frequent itemset tăng nhanh và kéo theo chi phí thời gian, bộ nhớ tăng theo. Điều này đặc biệt rõ trên `retail`, nơi số lượng mẫu tăng từ 16 lên 2691 khi minsup giảm từ 5% xuống 0.2%.
- **So với SPMF:** SPMF vẫn là cài đặt tham chiếu mạnh và ổn định, nhưng cài đặt Julia của nhóm cho thấy khả năng cạnh tranh tốt, thậm chí vượt SPMF ở một số cấu hình như `accidents` tại minsup 70% và 75%.

4.10 Điểm mạnh, điểm yếu của thuật toán và hướng tối ưu tiếp theo

4.10.1 Điểm mạnh của thuật toán H-Mine

- H-Mine tránh sinh ứng viên theo kiểu Apriori, do đó giảm được chi phí bùng nổ candidate trên nhiều trường hợp dữ liệu có nhiều mẫu phổ biến.
- Thuật toán sử dụng H-struct và hyper-link để biểu diễn projected database theo cách *ảo*, nhờ đó tiết kiệm bộ nhớ hơn so với việc tạo projected database vật lý ở mỗi bước đệ quy.
- H-Mine đặc biệt phù hợp với dữ liệu thưa, nơi mức độ chia sẻ prefix không đủ lớn để FP-tree phát huy tối đa hiệu quả nén.
- Cấu trúc của H-Mine cho phép kết hợp linh hoạt với chiến lược khác trên dữ liệu dày, chẳng hạn chuyển sang FP-tree ở các projected database đủ đậm đặc.

4.10.2 Điểm yếu của thuật toán H-Mine

- Hiệu năng của H-Mine vẫn phụ thuộc mạnh vào minsup: khi minsup giảm, số lượng frequent itemset tăng nhanh làm thời gian chạy và bộ nhớ tăng đáng kể.
- Trên dữ liệu rất dày hoặc projected database có nhiều prefix chung, H-Mine thuần túy có thể không tận dụng được khả năng nén tốt như FP-tree.

- Thuật toán vẫn phải đối mặt với hiện tượng bùng nổ số lượng mẫu đầu ra khi output size rất lớn; đây là hạn chế chung của bài toán frequent itemset mining.
- Việc lựa chọn thời điểm chuyển từ H-struct sang cấu trúc khác trên dữ liệu dày không đơn giản; nếu điều kiện chuyển chưa phù hợp thì overhead có thể làm giảm lợi ích kỳ vọng.

4.10.3 Hai hướng tối ưu cụ thể có thể áp dụng tiếp theo

1. **Tối ưu quản lý bộ nhớ trong đệ quy:** sử dụng buffer pool hoặc cơ chế tái sử dụng header table và các mảng tạm theo độ sâu đệ quy để giảm thêm allocation.
2. **Cải tiến chiến lược xử lý projected database dày:** thiết kế điều kiện chuyển sang FP-tree hoặc cấu trúc khác một cách thích nghi hơn, dựa đồng thời trên mật độ, số transaction và số item cục bộ.

5 Ứng dụng thực tế

5.1 Tổng quan bài toán

Phân tích giỏ hàng (Market Basket Analysis) là một trong những ứng dụng kinh điển và có giá trị thực tiễn cao nhất của khai phá dữ liệu. Bài toán đặt ra: từ lịch sử giao dịch của siêu thị, tìm ra các **luật kết hợp** (association rules) dạng $X \Rightarrow Y$, nghĩa là "khách hàng mua tập sản phẩm X thường có xu hướng mua thêm tập sản phẩm Y trong cùng một lần mua sắm". Kết quả giúp nhà bán lẻ đưa ra quyết định về bố trí kệ hàng, thiết kế chương trình khuyến mãi, và xây dựng hệ thống gợi ý sản phẩm.

5.2 Tập dữ liệu Groceries

Thực nghiệm sử dụng tập dữ liệu `Groceries`, một bộ dữ liệu giao dịch siêu thị thực tế được thu thập trong vòng một tháng.
Tập dữ liệu có đặc điểm thưa (sparse): trung bình mỗi giao dịch chứa khoảng 4–5 sản phẩm trong tổng số 169 loại, phù hợp với cấu trúc linked-list của H-Mine.

5.3 Các độ đo đánh giá luật kết hợp

Cho luật $X \Rightarrow Y$ với $X \cap Y = \emptyset$, các độ đo được định nghĩa như sau:

Table 5.1: Thông tin tập dữ liệu Groceries

Thuộc tính	**Giá trị**
Số giao dịch	9,835
Số loại sản phẩm	169
Định dạng	TXT (mỗi dòng = 1 giao dịch, item cách nhau bằng dấu phẩy)
Ví dụ	`whole milk, butter, yogurt, rice`

$$\text{support}(X \Rightarrow Y) = \frac{|\{t \in \mathscr{D} : X \cup Y \subseteq t\}|}{|\mathscr{D}|} \tag{5.1}$$

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)} = P(Y \mid X) \tag{5.2}$$

$$\text{lift}(X \Rightarrow Y) = \frac{\text{confidence}(X \Rightarrow Y)}{\text{support}(Y)} = \frac{P(X \cup Y)}{P(X) \cdot P(Y)} \tag{5.3}$$

$$\text{conviction}(X \Rightarrow Y) = \frac{1 - \text{support}(Y)}{1 - \text{confidence}(X \Rightarrow Y)} \tag{5.4}$$

Ý nghĩa của từng độ đo:

- **Support**: Tần suất xuất hiện đồng thời của X và Y. Lọc ra các luật có ý nghĩa thống kê, tránh kết luận từ quá ít quan sát.
- **Confidence**: Xác suất có điều kiện $P(Y \mid X)$. Đo mức độ tin cậy của luật.
- **Lift**: Tỉ số giữa confidence thực tế và confidence kỳ vọng nếu X, Y độc lập. Lift > 1 cho thấy X và Y có quan hệ thuận chiều; lift $= 1$ là độc lập; lift < 1 là quan hệ nghịch chiều. Lift là độ đo quan trọng nhất để xếp hạng luật vì loại trừ ảnh hưởng của sản phẩm phổ biến.
- **Conviction**: Đo mức độ phụ thuộc của Y vào X. Conviction càng cao thì luật càng chắc chắn, ít phụ thuộc vào ngẫu nhiên.

5.4 Cài đặt và quy trình thực nghiệm

Quy trình được cài đặt trong hai module Julia:

- `src/algorithm/rules.jl`: Module tái sử dụng định nghĩa struct `AssocRule`, hàm `generate_rules()` sinh luật từ tập frequent itemsets (hỗ trợ cả single và multi consequent), hàm `top_rules_by_lift()` và `print_rules()`.
- `src/market_basket_groceries.jl`: Pipeline hoàn chỉnh gồm đọc dữ liệu TXT, chạy H-Mine tối ưu để thu thập frequent itemsets, gọi module `Rules` để sinh luật và xuất kết quả.

Tham số thực nghiệm:

Table 5.2: Tham số thực nghiệm Market Basket Analysis

Tham số	**Giá trị**	**Lý do chọn**
minsup	1% (99 giao dịch)	Đủ ý nghĩa thống kê, không bỏ sót sản phẩm ít phổ biến
minconf	20%	Cho phép khám phá luật đa dạng, lift sẽ lọc thêm
Top-k	10	Tập trung vào các luật có giá trị thực tiễn cao nhất

Kết quả quá trình thực hiện:

- Frequent itemsets tìm được: **333**
- Tổng số luật thỏa conf $\geq$ 20%: **234** luật
- Thời gian chạy H-Mine: $\approx$**0.02 giây**
- Lift trung bình: 1.805
  Lift cao nhất: 3.295
- Confidence trung bình: 33.1%
  Confidence cao nhất: 58.6%

5.5 Top-10 luật kết hợp theo Lift

Bảng 5.3 trình bày 10 luật có lift cao nhất được khai phá từ tập Groceries.
Nhận xét chung: Tất cả 10 luật đều có lift > 2.7, nghĩa là xác suất mua đồng thời X và Y cao hơn ít nhất 2.7 lần so với kỳ vọng nếu hai tập này mua sắm độc lập nhau. Đây là bằng chứng thống kê mạnh về sự tồn tại của mẫu hành vi mua sắm trong dữ liệu thực tế.

Table 5.3: Top-10 association rules theo lift (minsup = 1%, minconf = 20%)

#	Antecedent (X)		Consequent (Y)	Support	Confidence	Lift	Conviction
1	{citrus fruit, other vegetables}	⇒	{root vegetables}	1.04%	35.92%	3.2950	1.3904
2	{yogurt, other vegetables}	⇒	{whipped/sour cream}	1.02%	23.42%	3.2671	1.2122
3	{tropical fruit, other vegetables}	⇒	{root vegetables}	1.23%	34.28%	3.1448	1.3557
4	{beef}	⇒	{root vegetables}	1.74%	33.14%	3.0404	1.3326
5	{citrus fruit, root vegetables}	⇒	{other vegetables}	1.04%	58.62%	3.0296	1.9491
6	{tropical fruit, root vegetables}	⇒	{other vegetables}	1.23%	58.45%	3.0210	1.9412
7	{whole milk, other vegetables}	⇒	{root vegetables}	2.32%	30.98%	2.8421	1.2909
8	{root vegetables}	⇒	{whole milk, other vegetables}	2.32%	21.27%	2.8421	1.1751
9	{butter}	⇒	{whole milk, other vegetables}	1.15%	20.73%	2.7706	1.1672
10	{whole milk, curd}	⇒	{yogurt}	1.01%	38.52%	2.7614	1.3997

5.6 Phân tích và ý nghĩa kinh doanh

5.6.1 Nhóm luật về rau củ (Luật 1–8)

Bảy trong mười luật đầu đều xoay quanh ba loại rau củ: *root vegetables* (cà rốt, khoai tây, củ cải), *other vegetables* (rau xanh tổng hợp), và *citrus/tropical fruit* (trái cây có múi/nhiệt đới). Đây là nhóm sản phẩm có mối liên hệ mạnh nhất trong toàn bộ tập dữ liệu.

Luật 1 {citrus fruit, other vegetables} ⇒ {root vegetables} (lift = 3.295): Đây là luật có lift cao nhất. Khi khách hàng đã bỏ trái cây có múi và rau xanh vào giỏ, xác suất họ mua thêm rau củ là 35.9%, cao gấp 3.3 lần so với tỉ lệ mua root vegetables của khách hàng bình thường (10.9%). Đây là nhóm khách hàng ăn uống lành mạnh, mua nguyên liệu nấu ăn tươi.

Luật 4 {beef} ⇒ {root vegetables} (lift = 3.040, conf = 33.1%): Một phần ba số khách mua thịt bò đều mua thêm rau củ trong cùng giao dịch, gấp 3 lần so với mức nền. Hành vi này phản ánh rõ ràng thói quen nấu các món hầm, súp, hay lẩu kết hợp thịt và rau của người tiêu dùng.

Luật 5 và 6: Confidence đạt trên 58%, cao nhất trong toàn bộ tập luật. Hơn một nửa số khách mua kết hợp *citrus/tropical fruit* + *root vegetables* đều mua thêm *other vegetables*. Ba loại rau củ–trái cây này cùng nhau tạo thành một "bộ nguyên liệu bữa ăn chính" điển hình.

Luật 7 và 8 tạo thành một cặp đối xứng thú vị: cùng là luật giữa {whole milk, other vegetables} và {root vegetables}, với lift bằng nhau (2.842). Điều này xác nhận rằng ba sản phẩm này có mức độ đồng xuất hiện cao và đối xứng, không phụ thuộc vào chiều của luật.

5.6.2 Luật về sản phẩm từ sữa (Luật 2, 9, 10)

Luật 2 {yogurt, other vegetables} ⇒ {whipped/sour cream} (lift = 3.267): Sự kết hợp giữa sữa chua và kem chua cùng rau xanh gợi ý đây là nhóm khách hàng có thói quen ăn uống Tây Âu, thường dùng kem chua làm sốt hay topping.

Luật 9 {butter} ⇒ {whole milk, other vegetables} (lift = 2.771): Bơ đi kèm sữa tươi và rau là bộ nguyên liệu nấu ăn quen thuộc. Đây là luật có consequent nhiều hơn 1 sản phẩm, chỉ xuất hiện nhờ tham số `single_consequent = false` trong cài đặt.

Luật 10 {whole milk, curd} ⇒ {yogurt} (lift = 2.761, conf = 38.5%): Khách hàng mua sữa tươi và phô mai tươi rất thường xuyên mua thêm sữa chua. Nhóm sản phẩm từ sữa (dairy) tạo thành một cluster mua sắm tự nhiên.

5.6.3 Nhận xét tổng hợp

Phân tích 10 luật trên cho phép nhận diện **hai nhóm khách hàng** chính:

- **Nhóm rau củ – thịt** (Luật 1, 3, 4, 5, 6, 7): Mua rau củ, trái cây tươi và thịt. Mua sắm theo mẫu nguyên liệu bữa chính. Nhóm này chiếm đa số trong dữ liệu.
- **Nhóm sản phẩm từ sữa** (Luật 2, 9, 10): Mua các sản phẩm từ sữa kết hợp với nhau. Khách hàng trung thành, mua định kỳ các sản phẩm có hạn sử dụng ngắn.

5.7 Đề xuất ứng dụng thực tiễn

Dựa trên các luật kết hợp khai phá được, có thể đề xuất các hành động kinh doanh cụ thể:

1. **Bố trí kệ hàng**: Đặt *root vegetables*, *other vegetables* và *citrus/tropical fruit* trong cùng một khu vực để tận dụng hiệu ứng mua sắm tuần hoàn giữa ba nhóm (Luật 1, 3, 5, 6). Tương tự, nhóm sản phẩm từ sữa (whole milk, yogurt, curd, butter) nên được bố trí trên cùng dãy tủ lạnh (Luật 10).
2. **Combo khuyến mãi**: Thiết kế gói thịt bò kèm rau củ và rau xanh, hoặc gói sữa tươi kèm sữa chua và phô mai tươi với giá ưu đãi, tận dụng trực tiếp Luật 4 và Luật 10.
3. **Gợi ý sản phẩm**: Tích hợp vào ứng dụng mua sắm hoặc hệ thống POS: khi khách thêm *beef* vào giỏ hàng, hiển thị gợi ý "Khách hàng thường mua thêm: Root vegetables" (Luật 4, conf = 33.1%).

4. **Quản lý tồn kho**: Các sản phẩm trong cùng luật có xu hướng hết hàng cùng thời điểm. Bộ phận mua hàng nên lập lịch nhập kho đồng bộ cho nhóm rau củ–trái cây và nhóm sản phẩm từ sữa.

5. **Phân khúc khách hàng**: Dùng các luật như cơ sở để phân nhóm khách hàng, xây dựng chương trình tích điểm riêng cho nhóm rau củ–thịt và nhóm sản phẩm từ sữa.

Tài liệu tham khảo

[1] Pei, J., Han, J., Lu, H., Nishio, S., Tang, S., & Yang, D. (2007). *H-Mine: Fast and space-preserving frequent pattern mining in large databases*. IIE Transactions, 39(6), 593-605. Link: https://www.philippe-fournier-viger.com/spmf/Hmine.pdf

[2] Fournier-Viger, P. (2025). *SPMF: An Open-Source Data Mining Library*. Frequent Itemset Mining Algorithms. Link: https://www.philippe-fournier-viger.com/spmf/index.php?link=algorithms.php

[3] Han, J., Pei, J., & Yin, Y. (2000). *Mining frequent patterns without candidate generation*. Proceedings of the 2000 ACM SIGMOD. Link: https://hanj.cs.illinois.edu/pdf/sigmod00.pdf

[4] Agrawal, R., & Srikant, R. (1994). *Fast algorithms for mining association rules*. Proc. 20th VLDB Conf. Link: https://www.vldb.org/conf/1994/P487.PDF

[5] Lantz, B. (2019). *Groceries Dataset for Market Basket Analysis*. 1st Edition. GitHub Repository. Link: https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/groceries.csv

Phụ lục

1. Bảng đóng góp công việc

STT	Họ và tên	MSSV	Hạng mục chính	Đóng góp
1	Phạm Khánh Linh	23127083	Chương 1, 3, 4	100%
2	Võ Trung Hiếu	23127190	Chương 2, 3, 4	100%
3	Nguyễn Thành Lợi	23127408	Chương 1, 3, 5	100%

Table 5.4: Bảng phân công khối lượng công việc thành viên

2. Mã nguồn dự án (Source Code)

Toàn bộ mã nguồn, dữ liệu về thuật toán HMine và ứng dụng phân tích giỏ hàng của Lab 2 được lưu trữ công khai trên GitHub tại địa chỉ: https://github.com/chichic21039/Lab2-DataMining-HMine